

Model predictive pitch control with an agent-supervised learned residual:

the residual repairs the model, the agent supplies the reward’s vocabulary

[authors]

Preprint. September 15, 2026

Abstract

Model predictive control (MPC) of wind-turbine pitch is only as good as its model, and reward design for a learned controller is only as good as its designer. We study the combination on collective-pitch control of the NREL 5 MW turbine in OpenFAST: a linear-parameter-varying MPC whose cost is scaled correctly is the strongest controller on the metric set of a published RL pitch-control study (power MSE, generator-speed MSE, tower-base and blade-root fatigue), yet a 5 % error in its aerodynamic model halves its score and 15 % removes its advantage entirely. A bounded residual policy trained by PPO on top of the MPC, with a reward supervised every 30 episodes by a language-model agent under fork verification, adds about five points to the exact-model MPC on two disjoint held-out wind sets with every metric improved (tower fatigue reduction from 16 to 23–25 %), and restores the mismatched MPC from 4.5 to 13–18—above the exact-model MPC alone. A controlled study of the supervision itself, run with the same protocol on the residual’s conventional base (the gain-scheduled PI), shows what the agent contributes: nothing through numeric levers (reward weights, hyper-parameters, where it ties random search), and, through the reward’s expression, a vocabulary of term shapes that reaches a balanced solution no weight search reaches—which a random draw from the same vocabulary, verified by the same loop, reaches as well; the loop is necessary, the objective stated in the prompt is not, and the agent’s effect on the level of the objective is within seed noise. We distil the findings into design rules: put learning where the model fails, put the agent on structure, verify against the true objective, one lever per agent, and report the composition of a solution alongside its mean.

1 Introduction

Above rated wind speed a turbine’s collective pitch has to regulate rotor speed and power while limiting fatigue of the tower and the blades. Model predictive control (MPC) handles the trade-off explicitly and is a standard reference for this problem [1–5]; reinforcement learning (RL) has been proposed as a model-free alternative [6–9]. The two are usually compared, rarely combined, and the comparison hides two dependencies: the MPC depends on its model, and the RL controller depends on its reward. Language-model (LLM) agents now write and tune rewards—Eureka evolves reward code against a task score [10], Language-to-Rewards and Text2Reward map instructions to reward parameters and programs [11, 12], DrEureka adds domain-randomisation design [13], and agentic reward designers close the loop with semantic feedback [14]—which raises a third question: what does the agent actually contribute, beyond a search that any verification loop could run?

This paper answers the three questions on one benchmark with one protocol.

Does a learned residual add to a good model-based controller, and does it repair the model? We put a bounded PPO residual on top of an LPV-MPC whose cost has been scaled so that its tower term works, and train it under a supervised reward. It adds about five points

of the paper’s metric-set objective to the exact-model MPC with every metric improved, and it restores an MPC whose aerodynamic model is 5 % off from 4.5 to 13–18—above the exact-model MPC alone (§3.1).

What does the agent contribute to the residual’s reward? On the MPC base, about one point between a fixed and an agent-written reward. To see what that point is made of we run the supervision study on the residual’s conventional base, the gain-scheduled PI, where the residual has more to do: three levers (reward weights, hyper-parameters, reward expression), each with a random-search control, plus loop ablations (§3.2). Numeric levers are worth nothing on a tuned default; the reward’s shape is the one lever that changes what the solution is made of; the loop is necessary, the prompt’s objective is not, and a random draw from the agent’s own vocabulary of shapes does as well as the agent.

What is the reference conditional on? The MPC’s lead over every RL arm on the PI base is real but requires an exact aerodynamic model; a 5 % error on the power coefficient halves it (§2.3). That fragility is the reason the residual has something to repair.

Contributions. (a) The combination of a correctly scaled LPV-MPC with a supervised learned residual, evaluated on two disjoint held-out wind sets: +5 points over the exact-model MPC, every metric positive, and a 5 % model error repaired. (b) A controlled protocol for agentic supervision of a learning controller—three levers, random-search controls, seed pairing, exact permutation tests, tuned fixed defaults—and its outcome: the agent’s contribution is a vocabulary of reward shapes made real by verification against the true objective, not the search and not the prompt. (c) A corrected model-based reference: the published verdict that MPC “loses tower fatigue” is a cost-scaling artefact; with $O(1)$ cost terms the same three-state MPC is positive on all four metrics, and its sensitivity to model error is quantified.

2 Setup

2.1 Plant, reference controller, wind

NREL 5 MW onshore turbine [15] in OpenFAST 4.2.1 [16], controlled by ROSCO 2.10.5 [17] through a patched ZeroMQ interface at the 10 ms control step. The ROSCO configuration is the gain-scheduled PI (GSPI) with peak shaving on and every other load-mitigation feature off, constant torque above rated; it is the pairing reference for every metric. Wind fields: TurbSim, 8, 12.5 and 15 m/s mean hub wind, turbulence intensity 8 %, 150 s episodes of which 130 s are scored. Realisations with TurbSim seeds 1–2 are the supervisor’s evaluation winds; seeds 3–6 and seeds 7–10 are two disjoint held-out sets (“wind seeds 3–6” and “wind seeds 7–10” below). Fatigue is the damage-equivalent load from rainflow counting [18, 19].

2.2 Objective and statistics

The objective is the metric set of [9] as one scalar,

$$J = \frac{1}{4} \sum_{m \in \{P, \omega, T, B\}} \text{clip}_{\pm 100}(\Delta m [\%]) - 20 \max(0, E_{\text{loss}} [\%] - 1), \quad (1)$$

the mean of the per-episode-clipped percentage reductions, relative to paired GSPI, of power MSE and generator-speed MSE on above-rated steps (labelled by hub wind, whatever the controller does) and of tower-base and blade-root fatigue, minus a penalty for energy loss beyond 1 %. Every selection (fork winner, best checkpoint, rollback) uses J ; every reported number is a held-out evaluation of the best checkpoint by the supervisor-side J . Seeds are paired across

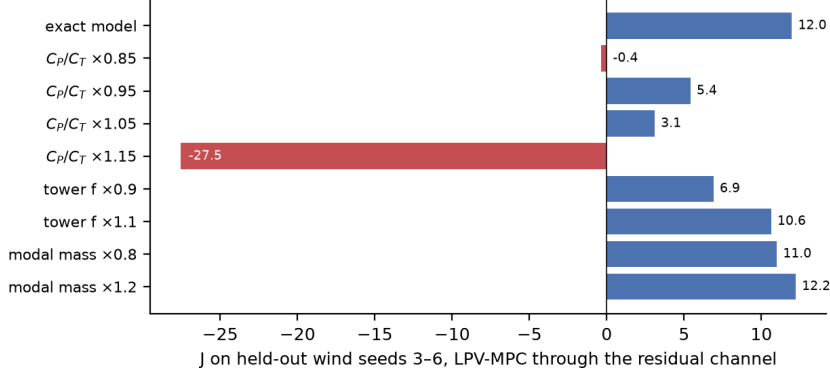


Figure 1: The MPC is conditional on its aerodynamic model. Held-out J of the J -selected LPV-MPC (through the residual channel) when only its internal model is perturbed; the plant is unchanged. Errors on the tower model are tolerated, errors on the power and thrust coefficients are not.

arms; differences are reported as mean, wins/ n , the exact two-sided sign-flip permutation p (floor $2/2^n$: 0.0625 at $n = 5$, 0.25 at $n = 3$) and a paired t -test. Arms with $n = 3$ are proofs of concept and are labelled as such.

2.3 The model-based controller and what it is conditional on

A three-state LPV-MPC (rotor, first tower fore-aft mode) linearised every 0.1 s from the C_p/C_T tables, horizon 2 s, OSQP [20] (3.6 ms per solve), pitch within the live peak-shaving floor and the rate limit, wind held at ROSCO’s low-passed estimate (no preview, unlike [3, 4]), torque native. Its cost has three terms—speed error, tower-top velocity, pitch increment—each scaled to $O(1)$ at its typical value (Appendix C). The scaling matters: with the speed error normalised by rated speed instead, the speed term is $O(10^{-5})$ at a typical error of 0.5 % while the tower term is $O(1)$, any tower weight above zero out-weighs regulation by four orders of magnitude, and the optimum feathers the rotor—which reproduces the published verdict that MPC “wins regulation and loses tower fatigue” (−17.6 % here). With $O(1)$ terms and weights selected on wind seeds 1–2 by J (the same rule as every learned arm), the MPC is positive on every metric: 12.4 / 14.1 on the two held-out sets with full pitch authority, 12.0 / 11.6 through the $\pm 2.9^\circ$ damped residual channel the RL uses.

The lead is conditional on the model (Fig. 1): tower frequency $\pm 10\%$ and modal mass $\pm 20\%$ cost 0–5 points and off-design turbulence (turbulence intensity 14 %, 22 %) nothing, but a $\pm 5\%$ error on the power and thrust coefficients takes the MPC to 5.4 / 3.1 and $\pm 15\%$ to −0.4 / −27.5. An identified model will not be exact; this is what a learned residual has to repair.

2.4 The learned residual

Two PPO [21] agents—one per operating region, selected by an oracle rule on ROSCO’s native pitch command—add a bounded collective-pitch offset ($|\Delta\beta| \leq 0.05$ rad, second-order damped) to a base command. On the *MPC base* the MPC runs inside the environment, the residual is added to its target, the pitch limits apply to the sum, and the MPC target is in the observation; with zero residual the controller is the wide-open MPC, so the episode-0 policy is the MPC (plus the untrained actor’s small constant offset) and the best checkpoint is never below it on the supervisor winds. On the *GSPI base* the residual sits on ROSCO’s PI command, the configuration used for the supervision study. Observations: generator-speed error and its rate, measured pitch, hub wind speed, blade-root out-of-plane moment, tower-top fore-aft acceleration, and on the MPC base the base target. $\gamma = 0.998$, GAE $\lambda = 0.98$ [22], 64×64 tanh networks, 300 training episodes per run; the critic’s regression target is standardised by a running estimate

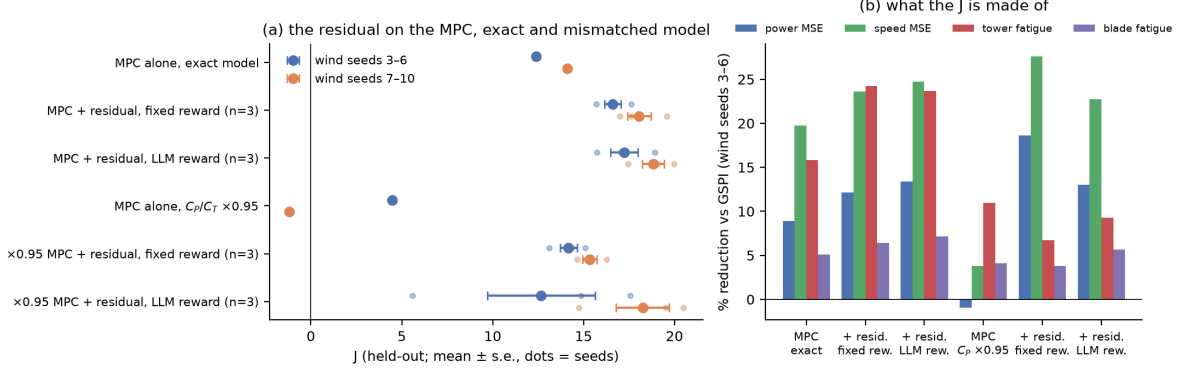


Figure 2: Residual on the MPC base. (a) Held-out J on both wind sets for the MPC alone and with the supervised residual, with an exact model and with the power/thrust coefficients 5% low in the MPC’s model (mean $\pm$ s.e., dots are seeds). (b) The four terms of J on wind seeds 3–6 for the same rows.

of the return scale [23]. The per-step reward is region-conditional,

$$r_t = \mathbf{1}[\text{Region 2}] w_P \left(\frac{P}{P_{\text{GSPI}}} - 1 \right) - \mathbf{1}[v_{\text{hub}} > v_{\text{rated}}] w_\omega \left(\frac{\delta\omega}{0.005} \right)^2 - \lambda_T \ell_T - \lambda_B \ell_B - \lambda_A (\Delta\beta/\kappa)^2, \quad (2)$$

with fatigue proxies ℓ_T, ℓ_B (growth of the 10s peak-to-peak range of tower acceleration and of blade-root moment, ≈ 1 per step under GSPI) and the speed term gated by the wind label—the subset on which the objective’s regulation metrics are scored (Appendix B). Its six coefficients and the two residual bounds are the *knobs*; their default is tuned on wind seeds 1–2 by J (Appendix A) and is the same on both bases.

2.5 The supervision loop and its levers

Every 30 episodes the policy is evaluated on wind seeds 1–2 and a supervisor may act. The agent (an OpenAI-compatible reasoning model in JSON mode, about five calls per run) receives the training diagnostics, the evaluation, the decision history with outcomes, and the objective’s definition; it returns $K = 3$ candidates. Each candidate is trained for a 30-episode fork from the same checkpoint, the forks are evaluated on wind seeds 1–2, and the best fork continues (*fork verification*). A guardrail rolls the run back to its best checkpoint when the objective drops by more than five points or energy loss exceeds 1%. Levers: the six **weights** of (2) (log-uniform random search as control); the PPO **hyper-parameters** (random search as control); the **reward expression**, a sandboxed Python expression over (region, wind label, speed error, power ratio, both fatigue proxies, actuation) that replaces (2), in the manner of Eureka [10], whose control is a random *structural* search over expressions drawn from a fixed grammar of the same term shapes with log-uniform coefficients, passed through the same loop. Fixed-knob training under the same guardrail is the *fixed-reward* baseline. Loop ablations: the agent asked once at the first decision and applied without a fork (*single-shot*); the agent told a different objective than the one that selects (a load-priority objective F with speed regulation as a penalised constraint—only what one agent is *told*, no results are reported under it).

3 Results

3.1 The residual on the MPC: it adds, and it repairs the model

Table 1 and Fig. 2 are the main result. On the exact-model MPC the residual adds +4 to +5 points on both held-out sets in every seed, with every metric improved and the tower-fatigue reduction rising from 16 to 23–25%; the composition is the MPC’s, made larger. On the MPC

Table 1: The residual on the MPC base (held-out; mean $\pm$ s.d.; composition on wind seeds 3–6 as power MSE / speed MSE / tower fatigue / blade fatigue, % reduction vs. GSPI). Energy is within $\pm 0.2\%$ of GSPI in every row.

controller	n	J seeds 3–6	J seeds 7–10	P / ω / T / B (seeds 3–6)	all four positive
LPV-MPC alone, exact model	—	12.40	14.10	8.9 / 19.8 / 15.8 / 5.1	yes
MPC + residual, fixed reward	3	16.60 ± 0.78	18.06 ± 1.10	12.1 / 23.6 / 24.2 / 6.4	3/3
MPC + residual, LLM-written reward	3	17.23 ± 1.30	18.84 ± 1.04	13.4 / 24.7 / 23.6 / 7.2	3/3
LPV-MPC alone, $C_P/C_T \times 0.95$ in its model	—	4.48	−1.0	−1.1 / 3.8 / 11.0 / 4.1	no
$\times 0.95$ MPC + residual, fixed reward	3	14.17 ± 0.82	15.35 ± 0.68	18.6 / 27.6 / 6.7 / 3.8	3/3
$\times 0.95$ MPC + residual, LLM-written reward	3	12.66 ± 5.13	18.25 ± 2.52	13.0 / 22.7 / 9.3 / 5.7	3/3

Table 2: The supervision study on the GSPI base (held-out J , mean $\pm$ s.d.; composition on wind seeds 3–6; “balanced” counts seeds on which all four metrics improve; paired differences against the tuned fixed reward with wins and exact p).

arm	n	J seeds 3–6	J seeds 7–10	P / ω / T / B	balanced	ΔJ vs. fixed (wins, exact p)
fixed reward (tuned)	5	4.86 ± 2.76	5.50 ± 2.09	17.4 / 19.0 / −17.6 / 0.7	0/5	—
LLM hyper-parameters	3	5.26 ± 2.39	7.13 ± 3.05	17.1 / 20.7 / −18.1 / 1.2	0/3	−0.5 / +0.3 (2/3, 1.0)
random hyper-parameters	3	4.93 ± 2.02	5.96 ± 2.17	15.1 / 17.0 / −13.2 / 0.8	0/3	−0.8 / −0.9 (2/3, 1.0)
LLM hyper-parameters + reward, one agent	5	6.37 ± 2.94	8.65 ± 2.60	15.9 / 16.3 / −7.6 / 0.9	0/5	+1.5 / +3.2 (4/5, 0.31 / 0.13)
LLM reward expression	5	5.30 ± 4.17	6.04 ± 4.28	6.8 / 10.1 / +1.5 / 2.7	3/5	+0.4 / +0.5 (3/5, 0.75 / 0.69)
LLM reward expression, told F , selected by J	5	7.00 ± 2.12	8.33 ± 2.38	9.5 / 14.6 / +0.9 / 3.0	3/5	+2.1 / +2.8 (4/5, 0.31 / 0.19)
random reward structure, same loop	3	6.13 ± 0.52	7.06 ± 1.35	9.5 / 10.5 / +2.1 / 2.5	1/3	+0.4 / +0.3 (2/3, 1/3; ≈ 1)
LLM reward expression, single-shot, no loop	5	4.97 ± 1.90	6.66 ± 2.75	11.6 / 13.3 / −5.5 / 0.5	1/5	+0.1 / +1.2 (2/5, 3/5; ≈ 1)

whose power and thrust coefficients are 5% low—alone at 4.5 / −1, its regulation lost—the residual restores 13–18, above the exact-model MPC alone: the residual learns the part of the plant the model gets wrong, on the same wind realisations it has never seen. Between a fixed reward and one written by the agent the difference on the MPC base is about one point on the exact model (one seed of the mismatched arm is poor); what the agent contributes to the residual’s reward is the subject of the next section.

3.2 What the agent contributes: a vocabulary, made real by verification

On the GSPI base the residual is the whole controller’s improvement, so the levers have room to matter; Table 2 and Fig. 3 give the outcome, all on the tuned default and both held-out sets.

Numeric levers are worth nothing. The hyper-parameter agent is worth −0.5 / +0.3 and random hyper-parameter search −0.8 / −0.9; the two proposers are indistinguishable and neither moves the fixed reward. The weight search (Appendix A) raises J by about two points over an untuned start and never keeps a candidate with a heavier fatigue weight: fatigue responds over tens of episodes, regulation within one fork, so a 30-episode verification horizon is myopic for that knob (Fig. 4b); a sweep of the tower weight confirms the balanced region is not on that axis at all ($\lambda_T = 3, 10, 30$: 1.4, −2.9, no learning).

The reward’s shape changes the composition, not the level. Every fixed-reward and hyper-parameter arm buys regulation with tower fatigue (−13 to −18%), the opposite of what a residual load controller is for. The reward-expression agent is the only arm whose mean tower fatigue improves and the only one with seeds on which all four metrics improve (3/5); its balanced seeds share a *saturated* speed term $\tanh(|\delta\omega|/0.005)$ on the wind-labelled subset and centred, bounded load terms $\tanh(\ell - 1)$ that penalise only above-baseline load (Appendix D). Its effect on the level of J is +0.4 / +0.5 (3/5, $p = 0.75 / 0.69$)—inside seed noise.

The loop is necessary; the prompt is not. Told F , a load-priority objective, the same agent writes the same shapes and, with J selecting its forks, reaches the balanced solution as

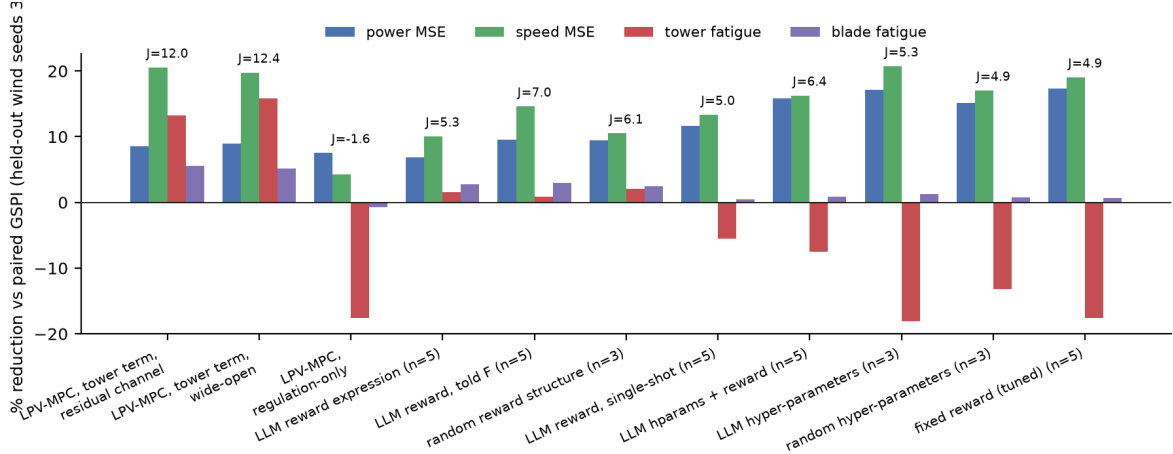


Figure 3: What each controller’s J is made of on the GSPI base (held-out wind seeds 3–6, arm means), with the MPC rows for scale. Every fixed-reward and hyper-parameter arm buys regulation with tower fatigue; the reward-expression agent, the random draw from its vocabulary, and the correctly scaled MPC are the rows with every term positive.

often (3/5) with the largest paired gain of any arm (+2.1 / +2.8, 4/5); its fork records show it choosing between candidates whose F and J disagreed, and J won each time because J selects. Asked once and applied blind, the same agent is the fixed reward (1/5 balanced). Pooled over the two looped arms the balanced solution appears in 6 of 10 seeds, in 1 of 5 without the loop and in 0 of 5 fixed-reward seeds.

The prior is a vocabulary. A random search over expressions drawn from a fixed grammar of the same term families the agent uses, verified by the same loop, lands where the agent lands: $6.13 \pm 0.52 / 7.06 \pm 1.35$, a positive mean tower fatigue, the balanced solution in 1 of 3 seeds, and the saturated forms kept. Once the vocabulary is written down, verification against the objective finds the balanced solution without an LLM; whether the agent finds it more reliably (6/10 against 1/3) is not separable at these seed counts. Combining the reward lever with the hyper-parameter lever in one agent raises the level (+1.5 / +3.2, 4/5) but never the composition (0/5 balanced): it buys regulation the way the hyper-parameter arms do.

4 Design rules

1. **Put learning where the model fails.** On the MPC base the residual adds five points and repairs a 5% model error; on the PI base the same residual and the same supervision reach a third of that.
2. **Put the agent on structural decisions; give numeric ones to random search.** No numeric lever, LLM or random, moved a sound default; the reward’s shape did, and a random draw from the agent’s vocabulary of shapes does as well. Use the agent to write the grammar, and a verified search to explore it.
3. **Verification against the true objective is the guarantee; the prompt is not.** The agent told the wrong objective produced the right solution because the loop selected on the true one; the agent without the loop produced the default.
4. **One lever per agent.** The combined agent gained a little level and lost the composition effect.
5. **Align the reward’s subsets and the cost’s scales with the objective before**

crediting the agent. A speed term gated on a different subset than the objective’s metric (Appendix B) and an MPC cost with terms four orders of magnitude apart each moved the comparison by more than any agent did.

6. **Report composition and reliability, not only means.** The agent’s contribution—a solution class, in a fraction of seeds—is invisible in a table of mean J .

5 Limitations

Seed budgets are three for the MPC-base arms and five for the core PI-base arms; no agentic difference in J is significant, and the MPC-base gains, while consistent in every seed, rest on $n = 3$. One plant, one training turbulence intensity, two held-out sets of four realisations. The MPC has no preview, a crude tower-state estimator and a two-second horizon; longer horizons were worse with this linearisation. The residual policy observes the simulator’s hub wind where the MPC uses ROSCO’s estimate; the model error studied is a scalar scaling of the aerodynamic coefficients, not an identified model. The agent is one reasoning model with one prompt family. An individual-pitch extension of the same supervisor was attempted and did not win on any of its levers; the boundary of the present claims is collective pitch.

References

- [1] Ervin A. Bossanyi. The design of closed loop controllers for wind turbines. *Wind Energy*, 3(3):149–163, 2000. doi: 10.1002/we.34.
- [2] Mostafa Soliman, Om P. Malik, and David T. Westwick. Multiple model predictive control for wind turbines with doubly fed induction generators. *IEEE Transactions on Sustainable Energy*, 2(3):215–225, 2011. doi: 10.1109/TSTE.2011.2153217.
- [3] Arne Koerber and Rudibert King. Combined feedback–feedforward control of wind turbines using state-constrained model predictive control. *IEEE Transactions on Control Systems Technology*, 21(4):1117–1128, 2013.
- [4] David Schlipf, Dominik Johannes Schlipf, and Martin Kühn. Nonlinear model predictive control of wind turbines using LIDAR. *Wind Energy*, 16:1107–1129, 2013. doi: 10.1002/we.1533.
- [5] Martin A. Evans, Mark Cannon, and Basil Kouvaritakis. Robust MPC tower damping for variable speed wind turbines. *IEEE Transactions on Control Systems Technology*, 23: 290–296, 2015.
- [6] Jesús Enrique Sierra-García and Matilde Santos. Exploring reward strategies for wind turbine pitch control by reinforcement learning. *Applied Sciences*, 10(21):7462, 2020. doi: 10.3390/app10217462.
- [7] Jingjie Xie, Hongyang Dong, and Xiaowei Zhao. Data-driven torque and pitch control of wind turbines via reinforcement learning. *Renewable Energy*, 215:118893, 2023.
- [8] Maud Coquelet et al. A reinforcement-learning approach for individual pitch control. *Wind Energy*, 2022. doi: 10.1002/we.2734.
- [9] [initials] Wang, Hongyang Dong, and Xiaowei Zhao. [title to be supplied]. *IEEE Transactions on Sustainable Energy*, 2026. Bibliographic details to be supplied by the authors before submission.

- [10] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.12931.
- [11] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, et al. Language to Rewards for robotic skill synthesis. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2306.08647.
- [12] Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2Reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.11489.
- [13] Yecheng Jason Ma et al. DrEureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2406.01967.
- [14] Hojoon Lee et al. RDA: Reward design agent for reinforcement learning, 2026. arXiv:2606.01672.
- [15] Jason Jonkman, Sandy Butterfield, Walter Musial, and George Scott. Definition of a 5-MW reference wind turbine for offshore system development. Technical Report NREL/TP-500-38060, National Renewable Energy Laboratory, 2009.
- [16] National Renewable Energy Laboratory. OpenFAST: an open-source wind turbine simulation tool. <https://github.com/OpenFAST/openfast>. version 4.2.1.
- [17] Nikhar J. Abbas, Daniel S. Zalkind, Lucy Pao, and Alan Wright. A reference open-source controller for fixed and floating offshore wind turbines. *Wind Energy Science*, 7:53–73, 2022.
- [18] Gunnstein T. Frøseth. fatpack: fatigue analysis in Python. <https://github.com/Gunnstein/fatpack>.
- [19] International Electrotechnical Commission. IEC 61400-1: Wind energy generation systems – part 1: Design requirements. edition to be fixed at submission.
- [20] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020. doi: 10.1007/s12532-020-00179-2.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
- [22] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1506.02438.
- [23] Hado van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1602.07714.

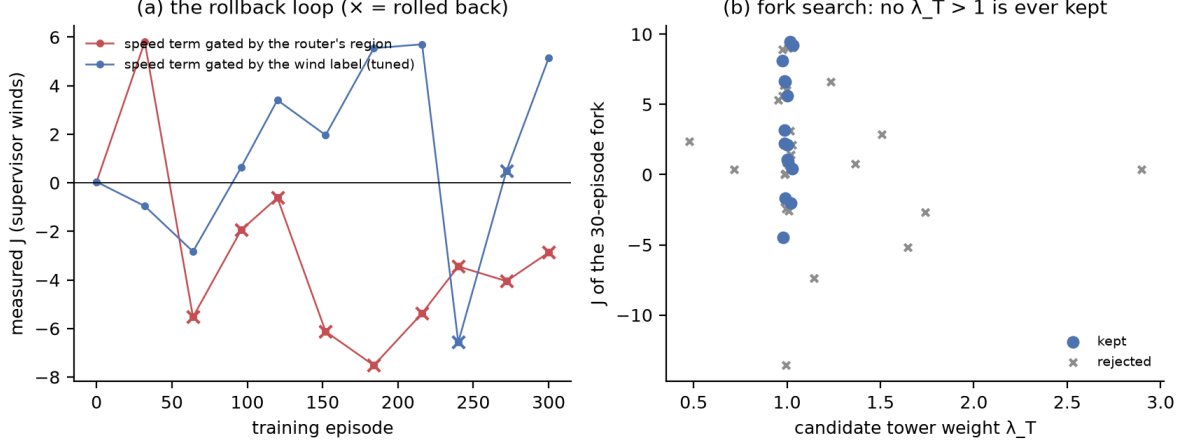


Figure 4: (a) Measured J on the supervisor winds at every decision of a fixed-reward run whose speed term is gated by the router’s region (red) and of the tuned default gated by the wind label (blue); crosses mark rollbacks. (b) Every fork candidate’s tower weight against the J of its 30-episode fork in the weight search; no candidate with $\lambda_T > 1$ is ever kept.

A Objective, metrics and the tuned default

The four metrics of (1) are computed per episode against the paired GSPI run on the same wind realisation: power MSE and generator-speed MSE relative to rated on the steps where hub wind exceeds rated (the 8 m/s episodes contribute no regulation term), and damage-equivalent loads of tower-base fore-aft moment and blade-root out-of-plane moment over all episodes. Each episode’s reduction is clipped to $\pm 100\%$ before averaging so that a single blown regulation episode cannot own the objective. Energy is the total over all episodes.

The tuned default of (2) is the per-knob geometric median of the best checkpoints of a three-seed random-search arm over the six knobs on the GSPI base (fork-verified, J -selected on wind seeds 1–2): $w_P = 220$, $w_\omega = 25.2$, $\lambda_T = 1$, $\lambda_B = 1.31$, residual bounds 0.074 rad (below rated) and 0.05 rad (above). That search raised held-out J from $4.60 \pm 2.38 / 5.96 \pm 2.02$ (untuned) to $6.70 \pm 2.12 / 7.89 \pm 2.21$ but never kept a candidate with $\lambda_T > 1$ (Fig. 4b). The tower-weight sweep from the tuned default (seed 0, wind seeds 3–6 / wind seeds 7–10): $\lambda_T = 1$: 5.0 / 5.9; 3: 1.4 / 3.7; 10: $-2.9 / -3.3$; 30: the policy never left its initial state. The same default is used unchanged on the MPC base.

B Why the reward’s speed term is gated by the wind label

The objective’s regulation metrics are scored on the steps where hub wind exceeds rated, whatever the controller does. If the reward’s speed term is instead gated by the router’s region, the transition steps at 12.5 m/s—router “below rated”, objective “above”—carry no speed penalty, and the policy is rewarded there for load reduction that the objective scores as regulation loss. Under that gating every fixed-reward run found an early optimum and then rolled back at every later decision (Fig. 4a): held-out J $1.59 \pm 0.87 / 0.97 \pm 1.37$ ($n = 3$). On that default, hyper-parameter supervision appears valuable ($+5.7 / +8.0$ for the LLM, $+5.8 / +7.5$ for random search, both by lengthening the credit window, GAE λ $0.98 \rightarrow 0.995$, and slowing the actor). Gating the speed term by the wind label removes the loop without any agent, and on that default the same hyper-parameter arms are worth nothing (§3.2). The aligned gating is used throughout the main text; the lesson is design rule 5.

C The MPC: cost scaling, selection, authority, robustness

The controller model is $J\dot{\omega} = T_{\text{aero}}(v - \dot{x}, \omega, \beta) - T_{\text{gen}}(\omega)$, $m\ddot{x} = F_{\text{thrust}}(v - \dot{x}, \omega, \beta) - c\dot{x} - kx$ with the first tower fore-aft mode at 0.324 Hz, modal mass 437 t and structural damping ratio 0.01, linearised at the operating point from the C_P/C_T tables every 0.1 s and discretised exactly; tower states are estimated by leaky double integration of the measured nacelle acceleration; the wind input is ROSCO’s estimate low-passed at 0.35 rad/s. The condensed QP over $N = 20$ steps minimises $\sum_k q (\delta\omega_k / (0.005 \omega_{\text{rated}}))^2 + q_t (\dot{x}_k / 0.2)^2 + r (\Delta\beta_k / 0.002)^2$ subject to the live pitch floor, the pitch limit and the rate limit. Selection on wind seeds 1–2 by J over $N \in \{20, 40\} \times r \in \{0.1, 0.3, 1, 3\} \times q_t \in \{0, 0.1, 0.3, 1, 3\} \times$ two wind-filter corners (82 points) gives $N = 20$, $r = 0.3$, $q_t = 3$; $N = 40$ is worse everywhere. Through the RL’s residual channel the same weights are re-selected. Table 3 collects the held-out rows and the model-error study; the model error is applied to the MPC’s internal model only.

Table 3: LPV-MPC (J -selected weights) on the held-out sets and under perturbation of its internal model only.

variant	J seeds 3–6	J seeds 7–10	P / ω / T / B (seeds 3–6)	note
full authority, exact model	12.40	14.10	8.9 / 19.8 / 15.8 / 5.1	base of §3.1
residual channel (± 0.05 rad, damped), exact model	11.97	11.61	8.5 / 20.5 / 13.2 / 5.6	
regulation-only cost ($q_t = 0$, unscaled)	−1.64	−8.08	7.5 / 4.2 / −17.6 / −0.7	the published verdict
full authority, $C_P/C_T \times 0.95$	4.48	−1.0	−1.1 / 3.8 / 11.0 / 4.1	base of §3.1
residual channel, $C_P/C_T \times 0.85$	−0.37		−3.8 / −2.8 / 2.6 / 2.5	regulation lost
residual channel, $C_P/C_T \times 0.95$	5.42		1.4 / 7.8 / 8.3 / 4.1	
residual channel, $C_P/C_T \times 1.05$	3.10		−9.1 / −0.1 / 17.0 / 4.5	
residual channel, $C_P/C_T \times 1.15$	−27.53		−58.2 / −74.2 / 16.5 / 5.7	regulation collapses
residual channel, tower frequency $\times 0.9 / \times 1.1$	6.91 / 10.63			
residual channel, modal mass $\times 0.8 / \times 1.2$	11.00 / 12.22			
residual channel, turbulence intensity 14% / 22% (15 m/s) / 18 m/s	10.43 / 12.31 / 22.45			off-design winds

D Reward expressions written by the agent

Best-checkpoint expressions of the reward-expression agent on the GSPI base (variables: **region** = router region, **region_w** = wind label, **d_wg** = relative speed error, **p_ratio** = power over paired GSPI power, **load_t**, **load_b** = fatigue proxies, **act** = actuation cost). Seed 0, told J :

```
180*tanh(15*(p_ratio-1))                                if region==0
-40*sqrt(tanh(abs(d_wg)/.005)**2 + tanh(60*abs(p_ratio-1))**2)  if region_w==1
-.9*tanh(log((2+load_t+load_b)/4)) - .1*tanh(act)
```

Seed 1, told F :

```
155*tanh(p_ratio-1)                                      if region==0
-26*tanh(abs(d_wg)/.005)                                if region_w==1
-(1.4 if region==0 else .5)*tanh(load_t)
-(1.0 if region==0 else 1.5)*tanh(load_b) - .1*tanh(act)
```

Seed 0, told F :

```
130*tanh(6*(p_ratio-1))*(region==0) - 24*tanh(d_wg/.005)**2*(region_w==1)
+ 1.0*tanh((1-load_t)/(1+load_t)) + 1.1*tanh((1-load_b)/(1+load_b)) - .04*tanh(act)
```

The random structural control samples expressions of the same families with log-uniform coefficients spanning the ranges the agent used. Speed terms, in $x = \delta\omega/0.005$: x^2 , $|x|$, $\tanh|x|$, $\tanh(x^2)$, $\sqrt{|x|}$. Load terms, in $\ell \in \{\ell_T, \ell_B\}$: ℓ , $\min(\ell, 3)$, $\tanh(\ell - 1)$, $\tanh \ell$, $\max(0, \ell - 1)$, $\log(1 + \ell)$. An optional power-deviation term on the wind-labelled subset.

E Paired differences and the full controller list

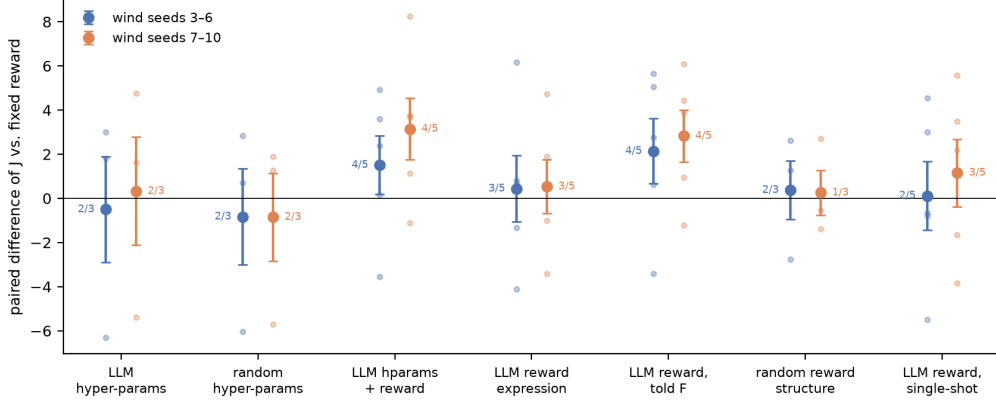


Figure 5: Seed-paired difference of J between each supervised arm and the tuned fixed reward on the GSPI base, both held-out wind sets (mean $\pm$ s.e., dots are seeds, labels are wins/ n).

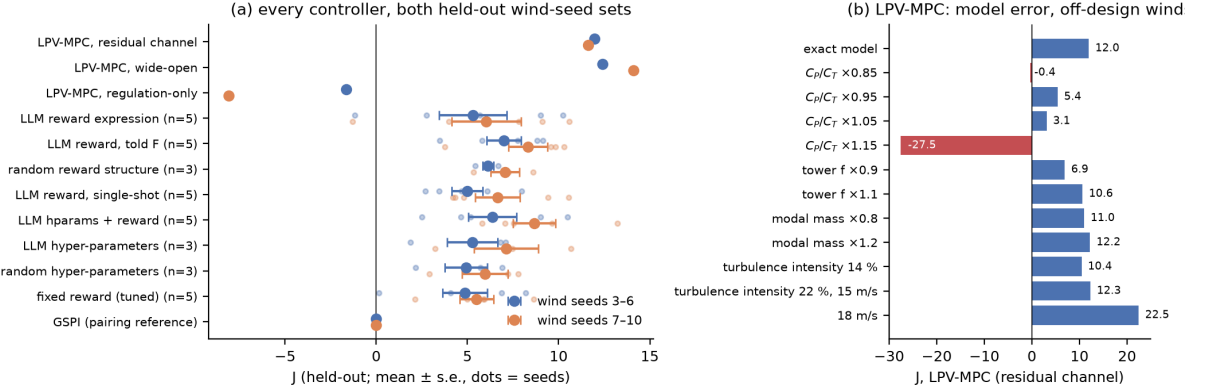


Figure 6: (a) J of every GSPI-base controller and the MPC variants on both held-out sets. (b) The MPC through the residual channel under model error and off-design winds (the data of Fig. 1, with the wind classes).

F Compute and reproducibility

One 300-episode training run takes 65–85 minutes on 16 CPU cores (8 OpenFAST workers, two runs in parallel); the MPC in the worker adds 3.6 ms per 0.1 s solve. A supervised run makes about five LLM calls. All runs write rolling checkpoints and can be paused and resumed; every number in this paper is regenerated by the scripts in the repository from the run artefacts, and the figures by a single script.